

NORMALIZATION

Normalization is the technique of **decomposing a large relation into smaller relations** such that:

- Data repetition(Redundancy) is minimized
- Dependencies are preserved
- Database becomes efficient and well-structured

Normalization generally involves splitting a table into multiple ones, which must be linked each time a query is made requiring data from the split tables.

Problems Without Normalization

Student-Course Table:

StudentID	StudentName	Course	Teacher
101	Amit	DBMS	Raj
101	Amit	CN	Neha
102	Simran	DBMS	Raj
103	Rahul	AI	Priya

Insertion Anomaly:

If a new course is introduced but no student has enrolled in it yet, we cannot store the course and teacher information because we need a student id record to insert.

Update Anomaly:

If the teacher for the DBMS course changes, we must update it in multiple rows (for every student enrolled in DBMS). If one row is missed, the database becomes inconsistent.

Deletion Anomaly:

If all students enrolled in the AI course drop it, we lose the course information completely, including the teacher assigned to that course.

Data Redundancy:

The course name and teacher name are repeated for every student enrolled in the same course, causing unnecessary duplication of data.

Advantages of Normalization

- o Normalization helps to minimize data redundancy.
- o Greater overall database organization.
- o Data consistency within the database.
- o Much more flexible database design.
- o Enforces the concept of relational integrity.

Features of Database Normalization

- **Eliminates Data Redundancy:** Reduces repetition of the same data, improving efficiency.
- **Ensures Data Consistency:** Prevents contradictions by keeping data accurate and uniform.
- **Simplifies Data Management:** Breaks complex tables into simpler ones for easier storage and retrieval.
- **Improves Database Design:** Creates a well-structured, flexible, and maintainable database.
- **Avoids Update Anomalies:** Prevents insertion, deletion, and update problems in tables.
- **Standardizes Data Storage:** Ensures data is stored in a consistent and organized way.

Normal Forms in DBMS

Normal Form	Description
First Normal Form (1NF)	A relation is in 1NF if every attribute contains only single (atomic) values and there are no repeating groups.
Second Normal Form (2NF)	A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the whole primary key (no partial dependency).
Third Normal Form (3NF)	A relation is in 3NF if it is in 2NF and there is no transitive dependency for non-prime attributes. A relation is in 3NF if for every FD $X \rightarrow Y$, either X is a super key or Y is a prime attribute.
Boyce-Codd Normal Form (BCNF)	A relation is in BCNF if it is in 3NF and for every functional dependency $X \rightarrow Y$, X must be a super key.
Fourth Normal Form (4NF)	A relation is in 4NF if it is in BCNF and does not contain any multivalued dependency.
Fifth Normal Form (5NF)	A relation is in 5NF if it is in 4NF and cannot be further decomposed without loss of information (no join dependency).

FIRST NORMAL FORM (1NF)

A relation (table) is in **First Normal Form (1NF)** if:

- Every attribute contains only **atomic (single) values**
- There are **no multivalued attributes**
- Each row is unique
- There are no repeating groups in any row.

Conditions for 1NF

A table satisfies 1NF when:

1. Atomic Values

- Each cell must store only **one value**, not a list.

2. No Repeating Columns

- Columns like Phone1, Phone2, Phone3 are not allowed.

3. Unique Rows

- No duplicate records should exist.

Example: 1NF

Student Table:

StudentID	Name	Phone Numbers
101	Amit	9876, 9999
102	Neha	8888
103	Rahul	7777, 6666

Problem:

- Phone Numbers column contains multiple values

- This violates atomicity

So, this table is **NOT in 1NF**.

Convert Table into 1NF

We split multivalued attribute into separate rows:

StudentPhone Table (1NF)

StudentID	Name	Phone Number
101	Amit	9876
101	Amit	9999
102	Neha	8888
103	Rahul	7777
103	Rahul	6666

Now:

- Each attribute has a single value
- No multivalued column
- Table is in **1NF**

Example of Repeating Groups (Not in 1NF)

EmpID	Name	Project1	Project2
1	Raj	AI	DBMS
2	Simran	ML	CN

Problem:

- Project columns are repeating groups

So, NOT in 1NF.

Convert to 1NF

EmpID	Name	Project
1	Raj	AI
1	Raj	DBMS
2	Simran	ML
2	Simran	CN

Practice Question: Convert into 1NF

Consider a university stores student details in the following table:

StudentID	StudentName	Courses
101	Amit	DBMS, CN
102	Neha	AI
103	Rahul	ML, DBMS, OS

Task: Convert the above relation into First Normal Form (1NF).

Second Normal Form (2NF)

Second Normal Form (2NF) is based on the concept of **fully functional dependency**.

For a table to be in 2NF, it must first meet the following requirements

1. Meet 1NF Requirements: The table must first satisfy **First Normal Form (1NF)**

2. Eliminate Partial Dependencies: A partial dependency occurs when a non-prime attribute (not part of the candidate key) depends only on a part of a composite primary key, rather than the entire key.

By ensuring these steps, a table in 2NF is more efficient and less prone to errors during updates, inserts, and deletes.

What is Partial Dependency?

The FD (functional dependency) $A \rightarrow B$ happens to be a partial dependency if **B is functionally dependent on A, and also B can be determined by any other proper subset of A.**

A partial dependency would occur whenever a non-prime attribute depends functionally on a part of the given candidate key.

Example: 2NF

StudentCourse Table:

StudentID	CourseID	StudentName	CourseName
101	C1	Amit	DBMS
101	C2	Amit	CN
102	C1	Neha	DBMS

Candidate Key: (StudentID, CourseID)

Functional Dependencies:

StudentID $\rightarrow$ StudentName

CourseID $\rightarrow$ CourseName

Problem:

- StudentName depends only on StudentID
- CourseName depends only on CourseID

So:

Non-key attributes depend on part of the key, This is **Partial Dependency**

Thus, table is in 1NF and Not in 2NF

Conversion into 2NF (Decomposition)

To remove partial dependency, we split the table into smaller relations.

1.Student Table

StudentID	StudentName
101	Amit
102	Neha

FD: StudentID $\rightarrow$ StudentName

2. Course Table

CourseID	CourseName
C1	DBMS
C2	CN

FD: CourseID $\rightarrow$ CourseName

3. Enrollment Table

StudentID	CourseID
101	C1
101	C2
102	C1

Key = (StudentID, CourseID)

Now:

- Every non-key attribute depends on full key
- Partial dependency removed
- Table is in 2NF

Anomalies Removed by 2NF

- **Update Anomaly Removed:**

Course name stored only once in Course table.

- **Insertion Anomaly Removed:**

New course can be added without student enrollment.

- **Deletion Anomaly Removed:**

Deleting a student enrollment does not delete course info.

Practice Question: Convert into 2NF

Question 1: . A company maintains employee project information:

EmpID	ProjectID	EmpName	ProjectName
E1	P1	Raj	AI System
E1	P2	Raj	Web App
E2	P1	Simran	AI System

Functional Dependencies:

$\text{EmpID} \rightarrow \text{EmpName}$

$\text{ProjectID} \rightarrow \text{ProjectName}$

Task: Convert this table into **Second Normal Form (2NF)**.

Question 2: A company stores employee project data:

Employee_Project Table

EmpID	EmpName	Projects	DeptName
E1	Raj	AI, Web	IT
E2	Neha	DBMS	HR
E3	Amit	AI, DBMS	IT

FD's:

$\text{EmpID} \rightarrow \text{EmpName}$

$\text{Project} \rightarrow \text{DeptName}$

Tasks:

1. Convert into **1NF**
2. Then convert into **2NF**

Question 3: A training institute maintains:

StudentID	CourseID	StudentName	CourseName	Fees
S1	C1	Amit	DBMS	5000
S1	C2	Amit	AI	6000
S2	C1	Neha	DBMS	5000

FDs:

StudentID $\rightarrow$ StudentName

CourseID $\rightarrow$ CourseName, Fees

Tasks:

- Check if table is in 1NF
- Convert into 2NF by removing partial dependency

Third Normal Form(3NF)

A relation is in **Third Normal Form (3NF)** if it satisfies the following two conditions:

1. It is in **Second Normal Form (2NF)**: This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).
2. There is **no transitive dependency for non-prime attributes**: In simpler terms, **no non-key attribute should depend on another non-key attribute**. Instead, all non-key attributes should depend directly on the primary key.

A relation is in the third normal form, if there is no transitive dependency for non-prime attributes as well as it is in the second normal form. A relation is in 3NF if at least one of the following conditions holds in every non-trivial function dependency $X \rightarrow Y$.

- **X is a super key.**
- **Y is a prime attribute** (each element of Y is part of some candidate key).

Examples: 3NF

Example 1

Employee_Department Table:

EmpID	EmpName	DeptID	DeptLocation
E1	Raj	D1	Delhi
E2	Neha	D2	Mumbai
E3	Amit	D1	Delhi

Candidate Key: EmpID

Functional Dependencies:

EmpID $\rightarrow$ EmpName

EmpID $\rightarrow$ DeptID

DeptID $\rightarrow$ DeptLocation

Transitive Dependency Exists: So the table is in 2NF and not in 3NF

Convert into 3NF (Decomposition)

To remove transitive dependency, split the table:

1. Employee Table

EmpID	EmpName	DeptID
E1	Raj	D1
E2	Neha	D2
E3	Amit	D1

FD: EmpID $\rightarrow$ DeptID

2. Department Table

DeptID	DeptLocation
D1	Delhi
D2	Mumbai

FD: DeptID $\rightarrow$ DeptLocation

Now:

- No transitive dependency
- Relation is in **3NF**

Example 2

StudentInfo(StudentID, StudentName, DeptID, DeptName, DeptHead)

StudentID	StudentName	DeptID	DeptName	DeptHead
S1	Amit	D1	Computer Sci	Dr. Sharma
S2	Neha	D2	Mechanical	Dr. Verma
S3	Rahul	D1	Computer Sci	Dr. Sharma

Step 1: Candidate Key: StudentID

Step 2: Functional Dependencies

StudentID $\rightarrow$ StudentName

StudentID $\rightarrow$ DeptID

DeptID $\rightarrow$ DeptName

DeptID $\rightarrow$ DeptHead

As observed, Transitive Dependency Exists

Therefore, Not in 3NF

Convert into 3NF (Decomposition)

Student Table

Student(StudentID, StudentName, DeptID)

StudentID	StudentName	DeptID
S1	Amit	D1
S2	Neha	D2
S3	Rahul	D1

Key: StudentID

Here, DeptID is Foreign Key

Department Table

Department(DeptID, DeptName, DeptHead)

DeptID	DeptName	DeptHead
D1	Computer Sci	Dr. Sharma
D2	Mechanical	Dr. Verma

Key: DeptID

Now Table is in 3NF

PRACTICE QUESTIONS

Question 1

Relation:

Order(OrderID, CustomerName, CustomerCity, ProductID, ProductName)

FDs:

- OrderID $\rightarrow$ CustomerName
- OrderID $\rightarrow$ ProductID
- CustomerName $\rightarrow$ CustomerCity
- ProductID $\rightarrow$ ProductName

Check for 3NF ?

Question 2

Relation:

Library(BookID, BookName, AuthorID, AuthorName, AuthorCountry)

FDs:

- BookID $\rightarrow$ BookName
- BookID $\rightarrow$ AuthorID
- AuthorID $\rightarrow$ AuthorName, AuthorCountry

Check for 3NF?

BCNF(Boyce Codd Normal Form)

For BCNF the relation should satisfy the below conditions

- The relation should be in the **3rd Normal Form**.
- **X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.**

BCNF is a **stronger version of 3NF**.

Rule: A relation is in **BCNF** if:

- **For every functional dependency $X \rightarrow Y$, X must be a Super Key.**

Form	Condition
3NF	X is superkey OR Y is prime attribute
BCNF	X must be superkey only

Why do we need BCNF?

Because sometimes a table is in **3NF** but still has redundancy due to:

- multiple candidate keys
- dependency where the determinant is not a key

Question 1:

Relation:

StudentCourse(Student, Course, Instructor)

Data:

Student	Course	Instructor
Amit	DBMS	Raj
Neha	DBMS	Raj
Rahul	AI	Priya

Functional Dependencies:

1. (Student, Course) $\rightarrow$ Instructor
2. Instructor $\rightarrow$ Course

Key: (Student, Course)

Check 3NF: So 3NF condition holds.

Relation is in 3NF

Check BCNF

BCNF says: Instructor must be superkey.

But the Instructor alone is not a superkey (cannot determine Student)

So: Not in BCNF

Convert into BCNF

Decompose into two tables:

1. InstructorCourse

InstructorCourse(Instructor, Course)

Instructor	Course
Raj	DBMS
Priya	AI

Key = Instructor

2. StudentCourseInstructor

StudentInstructor(Student, Instructor)

Student	Course	Instructor
Amit	DBMS	Raj
Neha	DBMS	Raj
Rahul	AI	Priya

Key = (Student, Course)

Now both tables satisfy BCNF

Question 2:

Given:

Relation:

$R(A, B, C, D)$

Functional Dependencies:

1. $A \rightarrow B$
2. $C \rightarrow D$
3. $A \rightarrow C$

Find:

1. Candidate keys
2. Is relation in BCNF?
3. If not, decompose into BCNF.

SOLUTION

Step 1: Find Candidate Keys

We compute closure of A:

$A \rightarrow B$

$A \rightarrow C$

$C \rightarrow D$

Therefore,

$A^+ = \{A, B, C, D\}$

Since A determines all attributes of the relation,

A is a candidate key.

Now check if any other attribute can determine all attributes:

- $B^+ = \{B\}$
- $C^+ = \{C, D\}$

- $D^+ = \{D\}$

No other attribute determines all attributes.

Hence,

Candidate Key = $\{A\}$

Step 2: Check for BCNF

BCNF condition:

For every functional dependency $X \rightarrow Y$, X must be a super key.

Now check each FD:

1. $A \rightarrow B$
 A is candidate key $\rightarrow$ Satisfies BCNF
2. $A \rightarrow C$
 A is candidate key $\rightarrow$ Satisfies BCNF
3. $C \rightarrow D$
 C is NOT a super key $\rightarrow$ Violates BCNF

Therefore,

The relation is **NOT in BCNF** because of $C \rightarrow D$.

Step 3: Decomposition into BCNF

Since $C \rightarrow D$ violates BCNF, we decompose based on this dependency.

Decompose R into:

$R_1(C, D)$

$R_2(A, B, C)$

Step 4: Check BCNF for Decomposed Relations

Relation $R_1(C, D)$

Functional Dependency:

$C \rightarrow D$

C determines all attributes of R_1 .

Therefore, C is a candidate key.

R_1 is in BCNF.

Relation $R_2(A, B, C)$

Functional Dependencies in R_2 :

$A \rightarrow B$

$A \rightarrow C$

Closure of A:

$A^+ = \{A, B, C\}$

A is candidate key.

All dependencies have super key on LHS.

Therefore, R_2 is in BCNF.

1. **Candidate Key: A**
2. **The given relation is NOT in BCNF.**
3. **After decomposition into:**

$R_1(C, D)$

$R_2(A, B, C)$

Both relations are in BCNF.

PRACTICE QUESTIONS

Q1. Relation:

Library(BookID, BookName, AuthorID, AuthorName, PublisherID, PublisherName, PublisherCity)

FDs:

- $\text{BookID} \rightarrow \text{BookName}$
- $\text{BookID} \rightarrow \text{AuthorID}$
- $\text{AuthorID} \rightarrow \text{AuthorName}$
- $\text{PublisherID} \rightarrow \text{PublisherName}, \text{PublisherCity}$
- $\text{BookID} \rightarrow \text{PublisherID}$

Tasks:

1. Check whether the relation is in 2NF. If not, convert it into 2NF.
2. Check whether the relation is in 3NF. If not, convert it into 3NF.

Q2. Relation:

Hospital(PatientID, PatientName, DoctorID, DoctorName, DepartmentID, DepartmentName)

FDs:

- $\text{PatientID} \rightarrow \text{PatientName}$
- $\text{PatientID} \rightarrow \text{DoctorID}$
- $\text{DoctorID} \rightarrow \text{DoctorName}$
- $\text{DoctorID} \rightarrow \text{DepartmentID}$
- $\text{DepartmentID} \rightarrow \text{DepartmentName}$

Tasks:

1. Find the candidate key(s).
2. Check whether the relation is in BCNF.
3. If not, decompose the relation into BCNF.
4. Verify that all decomposed relations satisfy BCNF

Q3. Relation:

CourseRegistration(StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName)

FDs:

- $(\text{StudentID}, \text{CourseID}) \rightarrow \text{InstructorID}$
- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{InstructorID} \rightarrow \text{InstructorName}$
- $\text{InstructorID} \rightarrow \text{CourseID}$

- Find the candidate key(s).
- Check whether the relation is in 2NF. If not, convert it into 2NF.
- Check whether the relation is in 3NF. If not, convert it into 3NF.
- Check whether the relation is in BCNF. If not, decompose it into BCNF.